SISTEMA BUG TRACKING (FlowCheck)

1. Introduzione e Analisi dei Requisiti:

1.1 Descrizione del progetto:

FlowCheck è un tool per tracciare bug e ottimizzare il ciclo di vita del software attraverso una strutturata Gestione Progetti. La piattaforma permette al team di organizzare il lavoro tramite Ticket, classificandoli chiaramente come Bug (per le anomalie) o Feature (per le nuove implementazioni). Per garantire un flusso di lavoro trasparente, ogni ticket può essere assegnato a uno sviluppatore specifico e segue uno stato di avanzamento ben definito (Aperto, In Lavorazione, In Revisione, Confermato, Chiuso). La collaborazione è facilitata da un sistema di commenti integrato per ogni ticket, mantenendo tutta la comunicazione tecnica tracciata in un unico posto, un'o ttima base strutturale anche in vista di futuri cicli di test con gli utenti.

1.2 Attori del sistema:

Il sistema prevede tre tipologie di attori con permessi diversi:

1) **Amministratore del progetto:** Ha accesso completo al sistema. Può creare nuovi progetti, gestire i ticket (eliminarli, modificarli o aggiungerli), intervenire sui commenti, gestire gli accessi e partecipanti.
2) **Sviluppatore:** può lavorare sui progetti, gestire ticket (modificare lo stato) e aggiungere commenti.
3) **Reporter:** testa i progetti, apre ticket (bug) a progetti e aggiunge commenti.

1.3 Requisiti Funzionali:

- Autenticazione: sistema di login sicuro con distinzione dei ruoli (differenza tra chi è l'amministratore e chi è lo sviluppatore/reporter)
- Gestione ticket: Un ticket può essere aggiunto, modificato o eliminato al progetto o allo sviluppatore
- Gestione progetti: l'amministratore crea progetti e assegna sviluppatori ai progetti.
- Gestione commenti: commenti possono essere aggiunti dagli utenti a ticket di un progetto.

1.4 Requisiti non Funzionali:

- Portabilità: l'applicazione deve poter girare su qualsiasi sistema operativo, ciò è garantito tramite l'uso di Docker.
- Sicurezza: le password devono essere salvate in modo sicuro nel database tramite l'hashing e le API devono essere protette.
- Usabilità: il sistema deve avere un interfaccia user-friendly, utilizzabile da tutti.
- Scalabilità: frontend e backend non sono dipendenti tra loro, comunicando esclusivamente tramite l'uso di API.

2. Glossario del progetto:

TERMINE	DESCRIZIONE	SINONIMI
Reporter	Attore del sistema che testa il progetto e può aprire ticket (Bug)	Tester
Sviluppatore	Attore del sistema che può sviluppare nuovo codice	Developer
Admin	Attore del sistema che ha il massimo dei poteri nel sistema	Amministratore del progetto, Amministratore

3. Architettura del Sistema:

3.1 Pattern architetturale MVP (Model-View-Presenter):

FlowCheck segue il pattern architetturale MVP, per garantire una rigorosa separazione delle responsabilità tra la persistenza dei dati, la logica applicativa e l'interfaccia utente.

- Model (Backend PHP + MySQL): Rappresenta il dominio applicativo e il livello di persistenza. Isola e gestisce in sicurezza le operazioni critiche come i flussi di login, il controllo dei ruoli amministrativi e le operazioni relative ai ticket. È implementato tramite classi PHP che interagiscono con il database.
- View (HTML/CSS): Costituisce la componente presentazionale e passiva del sistema. Definisce il layout visivo senza integrare la logica di elaborazione dei dati.

- Presenter (JavaScript): Si occupa di intercettare gli eventi scatenati dall'utente con la View, invocare le API esposte dal Model ed elaborare i JSON.

3.2 Containerizzazione con Docker:

Per garantire la massima portabilità e standardizzare l'ambiente di sviluppo, FlowCheck è stata interamente conteinerizzata grazie al file docker-compose.yaml.

- db: Immagine ufficiale di MySQL 8.0, dedicato esclusivamente alla persistenza dei dati.
- Backend: Costruito a partire da un Dockerfile, questo container ospita il servizio web Apache accoppiato a PHP.
- Frontend: container Nginx che serve i file statici HTML/CSS/JS del frontend, esposto sulla porta 3000.

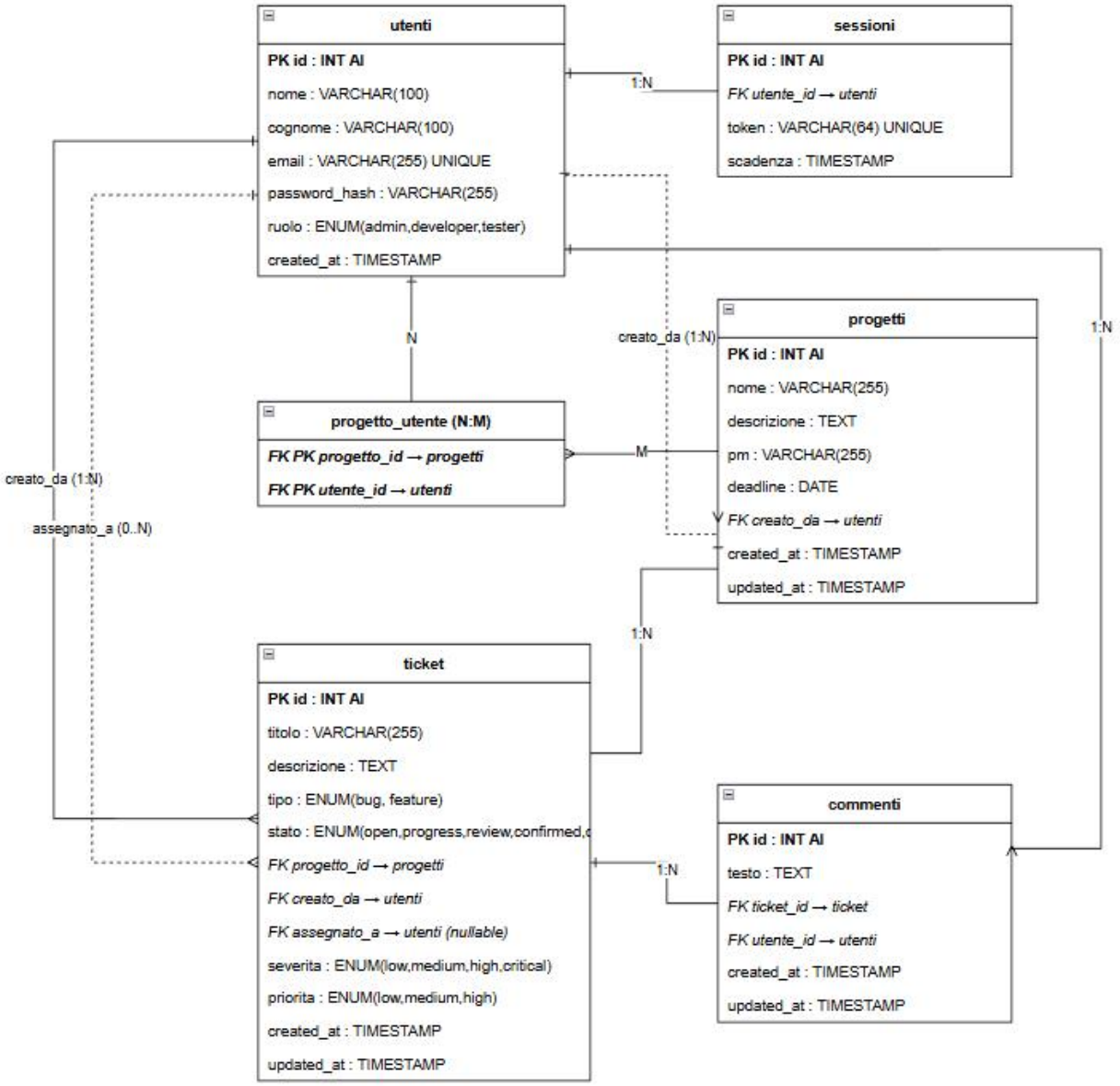
4. Scelte progettuali e Design Pattern:

Sono stati implementati i seguenti Design Pattern:

- **Factory Method:** Utilizzato per la creazione centralizzata delle istanze di ticket: a partire dai dati ricevuti, la factory produce un oggetto BugTicket o FeatureTicket, ciascuno con la propria logica di validazione.
  - **Implementazione:** TicketFactory.php (backend) e TicketFactory.js (frontend).
  - **Vantaggi:** Centralizza la logica di istanziazione e validazione; aggiungere un nuovo tipo di ticket richiede solo la creazione di una nuova classe concreta senza modificare i controller. Garantisce che tutte le comunicazioni tra Backend e Frontend mantengono un formato uniforme.
- **State Pattern:** Gestione del ciclo di vita dei ticket e delle transizioni di stato.
  - **Classi:** StatoAperto, StatoInLavorazione, StatoInRevisione, StatoConfermato, StatoChiuso, TicketStateContext, TicketStateInterface (backend). TicketStateContext, OpenState, ProgressState, ReviewState, ConfirmedState, ClosedState (frontend).
  - **Implementazione:** Ogni stato implementa i metodi canAdvance() e getNextStato(). Il contesto TicketStateContext incapsula lo stato corrente del ticket e delega a esso le decisioni sulle transizioni.

    - **Vantaggi:** Aggiungere un nuovo stato richiede solo la creazione di una nuova classe stato, senza modificare la logica del controller.
- **Singleton:** Gestione centralizzata della connessione al database.
    - **Classe:** Database (backend).
    - **Implementazione:** La classe mantiene un'istanza statica privata della connessione PDO, garantisce che durante l'intera richiesta http venga aperta una sola connessione al database.
    - **Vantaggio:** Evita di aprire connessioni multiple inutili al database durante la stessa richiesta.

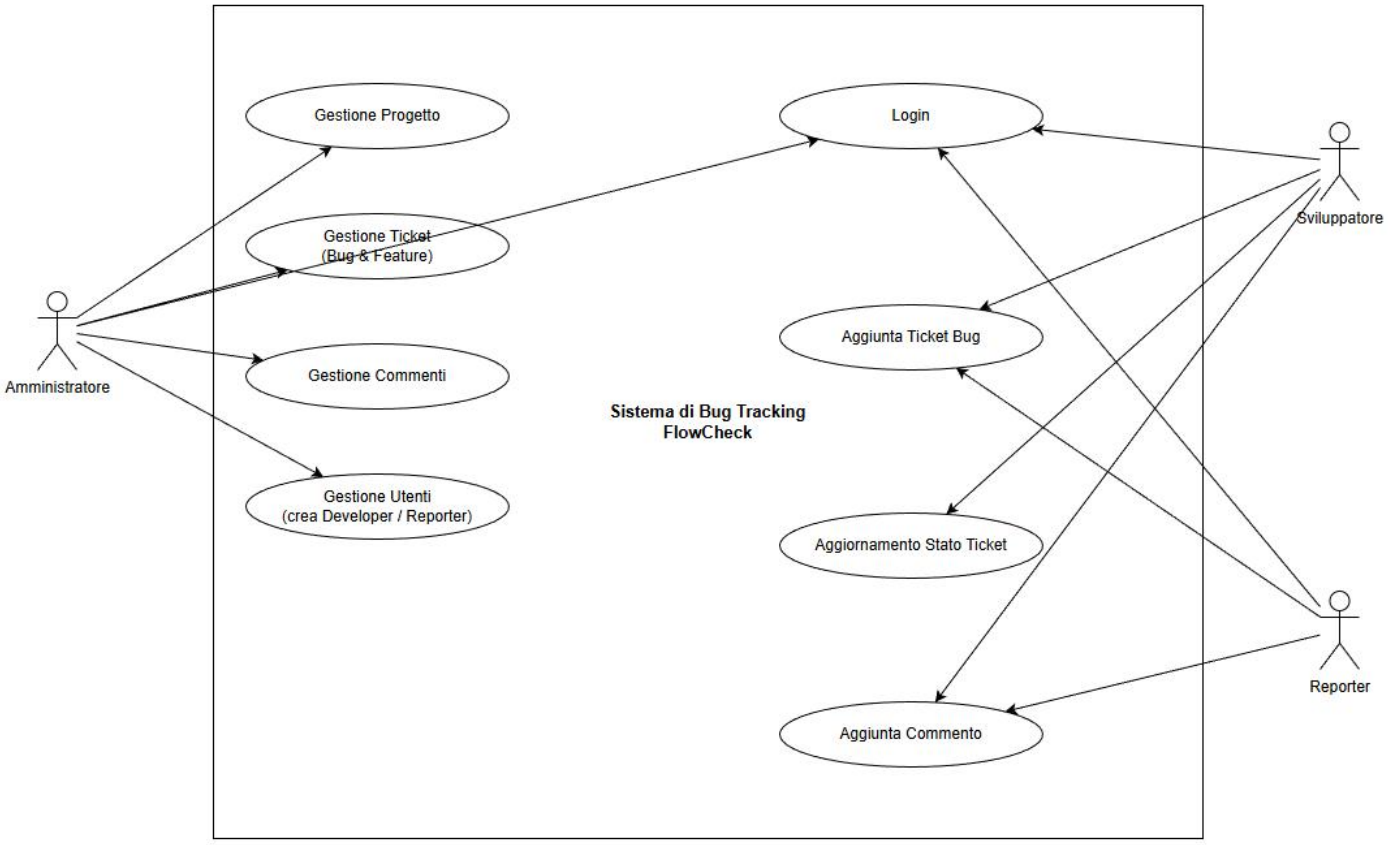
5. Database



6. Modellazione e Analisi del Sistema (UML)

6.1 Diagrammi dei Casi d'Uso

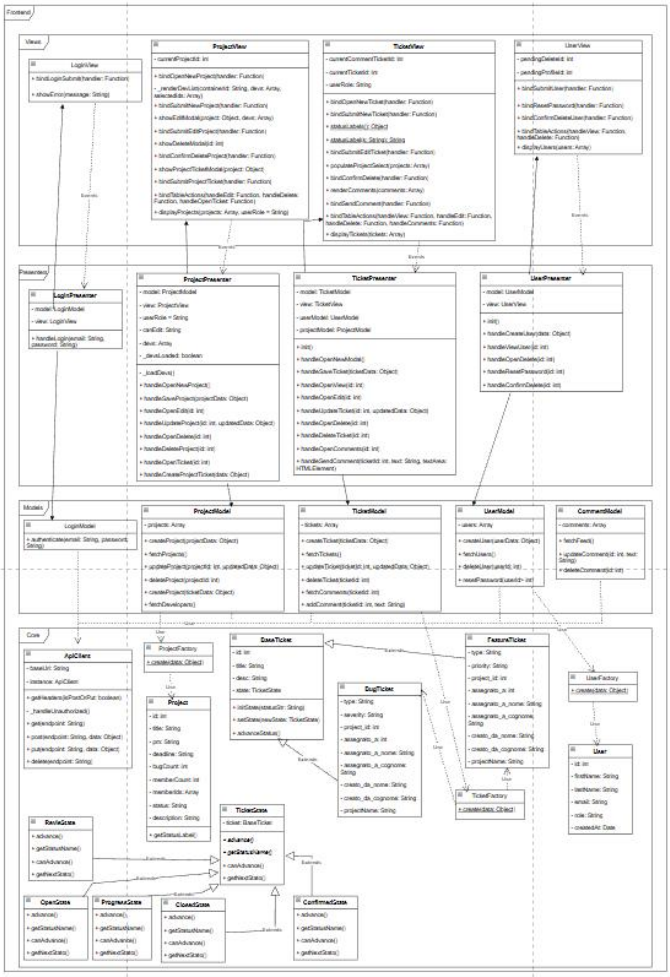
Il diagramma descrive le funzionalità offerte dal sistema e le interazioni con gli attori.

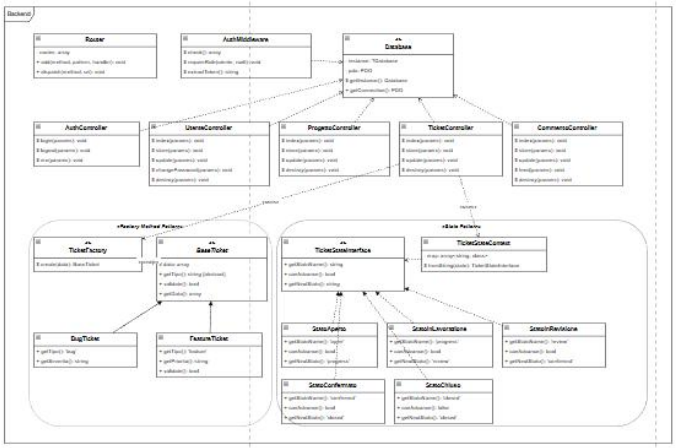


- **Attori**
  - **Amministratore:** L'utente con il massimo dei privilegi; gestisce il progetto, può creare Ticket (Bug & Feature), gestisce i commenti ai relativi ticket e gestisce gli accessi degli utenti ai progetti con i relativi permessi.
  - **Sviluppatore:** Utente che può lavorare ai progetti aggiungendo nuovo codice, può creare Ticket (Bug), Aggiornare lo stato dei Ticket e aggiungere commenti.
  - **Reporter:** Utente testa i progetti e reporta bug relativi ai progetti; aggiunge Ticket (Bug) e commenti.
- **Casi d'uso principali:**
  - Funzionalità di Login obbligatoria per tutti gli utenti della piattaforma.
  - Aggiunta di un nuovo progetto
  - Gestione dei Ticket da parte di tutti gli utenti con permessi diversi.

6.2 Diagramma delle Classi:

Questo diagramma offre una vista dettagliata sulla struttura del codice, evidenziando la separazione tra Frontend e Backend e l'uso del pattern MVP.





- **Frontend (Browser/JS):** Mostra l'interazione tra le View (gestione del DOM), i Presenter (logica di presentazione) e i Model (chiamate API).
- **Backend (PHP):** Mostra come gli i controller delle API si interfacciano con le altre classi per accedere ai dati. Per la struttura Descrive Router (entry point HTTP per tutte le richieste), AuthMiddleware (Autenticazione e controllo RBAC) e la classe Database (Singleton per la connessione PDO al database).

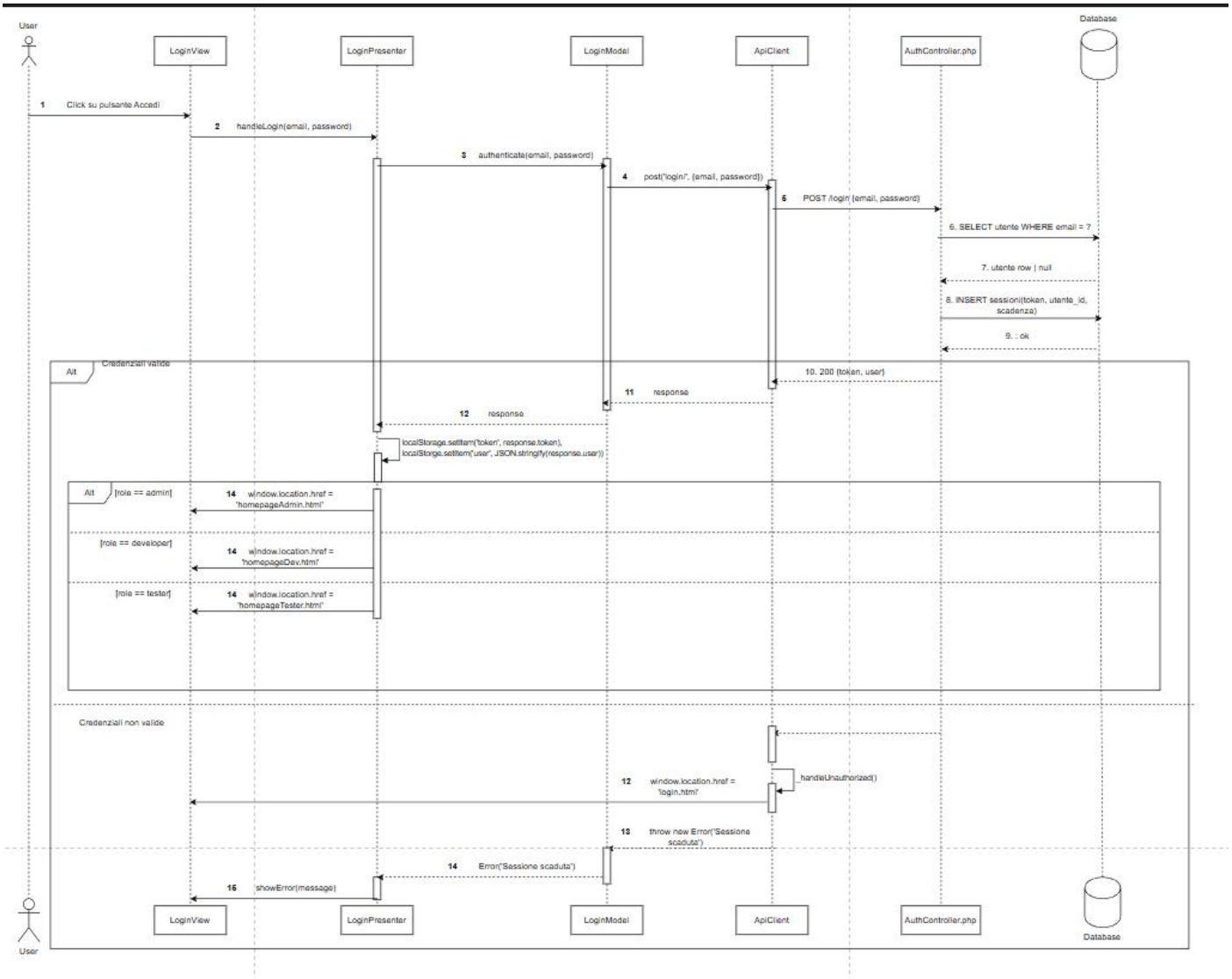
6.3 Diagramma di sequenza

I diagrammi di sequenza mostrano il flusso temporale dei messaggi tra i componenti per scenari critici:

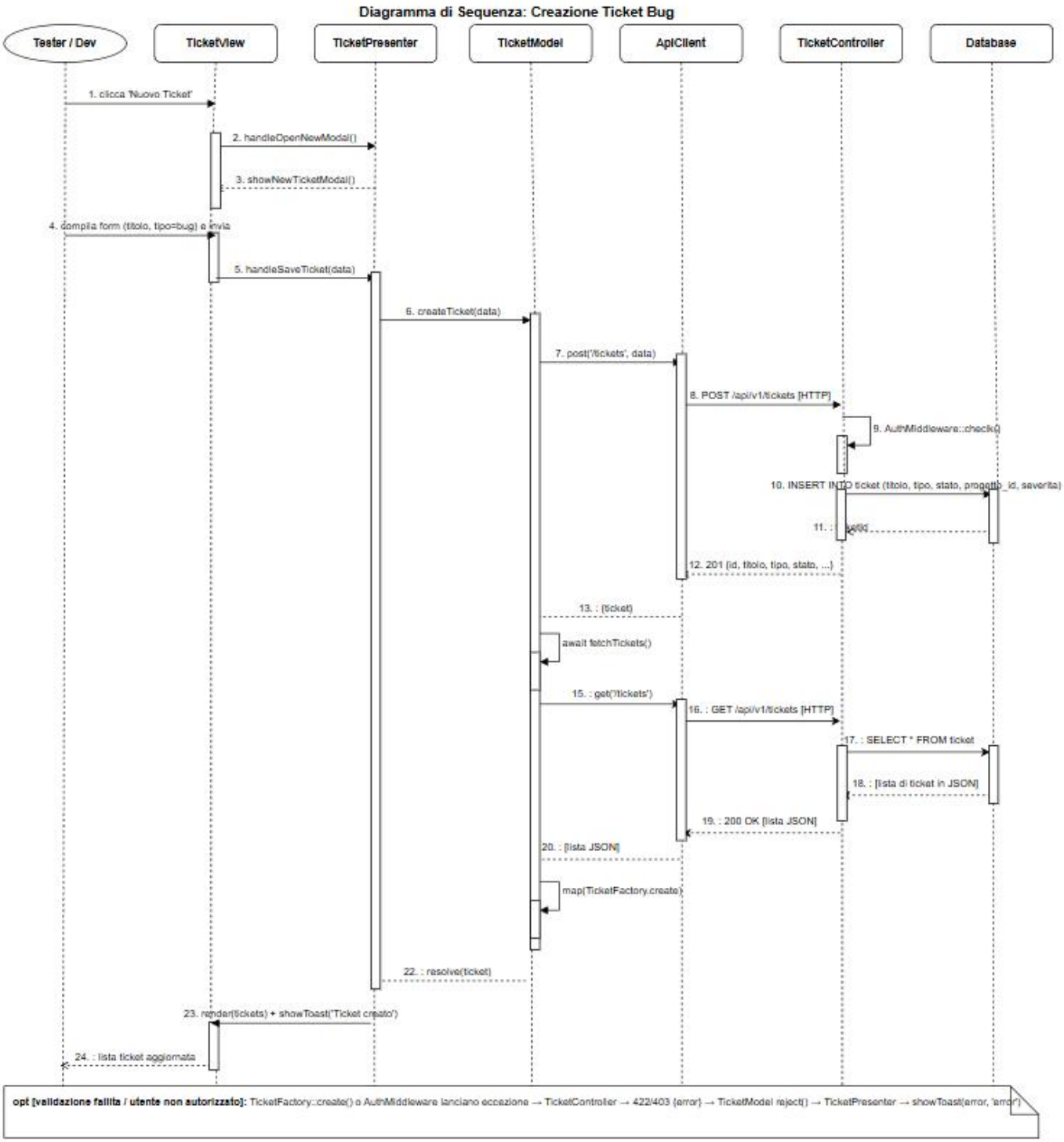
6.3.1 Scenario: Login Utente

Il flusso mostra l'autenticazione sicura:

1. L'utente inserisce le credenziali in LoginView.
2. LoginPresenter invia la richiesta a LoginModel.
3. LoginModel inoltra la richiesta al Backend tramite ApiClient.


User
LoginView
LoginPresenter
LoginModel
ApiClient
AuthController.php
Database
1 Click su pulsante Accedi
2 handleLogin(email, password)
3 authenticate(email, password)
4 post('login/', {email, password})
5 POST /login {email, password}
6. SELECT utente WHERE email = ?
7. utente row | null
8. INSERT sessioni(token, utente_id, scadenza)
9. : ok
Alt
Credenziali valide
10. 200 {token, user}
11 response
12 response
localStorage.setItem('token', response.token),
localStorge.setItem('user', JSON.stringify(response.user))
Alt
[role == admin]
14 window.location.href = 'homepageAdmin.html'
[role == developer]
14 window.location.href = 'homepageDev.html'
[role == tester]
14 window.location.href = 'homepageTester.html'
Credenziali non valide
_handleUnauthorized()
12 window.location.href = 'login.html'
13 throw new Error('Sessione scaduta')
14 Error('Sessione scaduta')
15 showError(message)

6.3.2 Scenario: Creazione Ticket



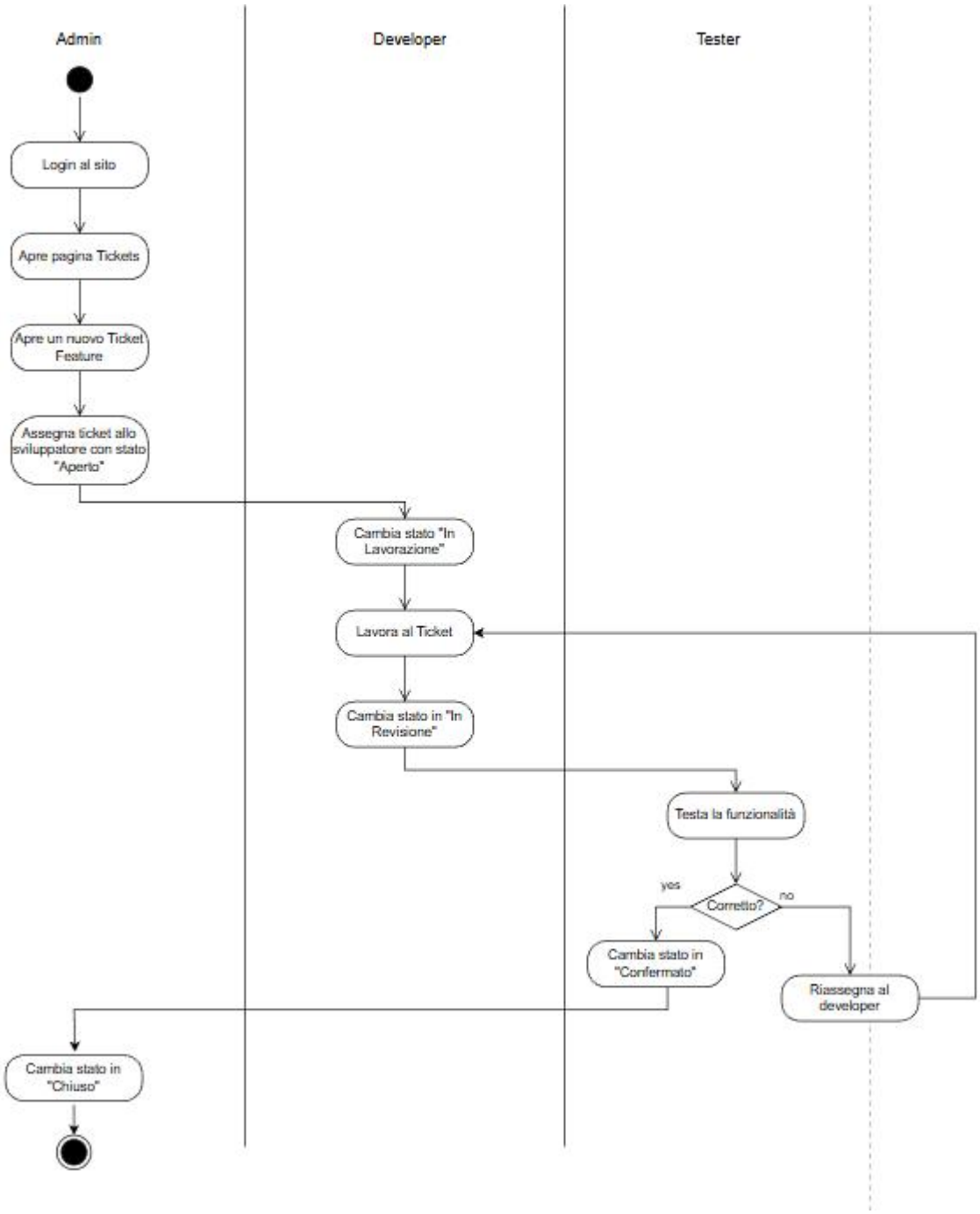
Il flusso mostra la creazione di un ticket:

1. L'utente clicca su nuovo ticket.
2. Compila il form con i dati del ticket.
3. TicketPresenter intercetta l'evento submit e chiama TicketModel.createTicket() con i dati del form.
4. TicketModel inoltra la richiesta al Backend tramite ApiClient.

5. L’utente vede il ticket creato nella pagina

6.4 Diagramma delle Attività

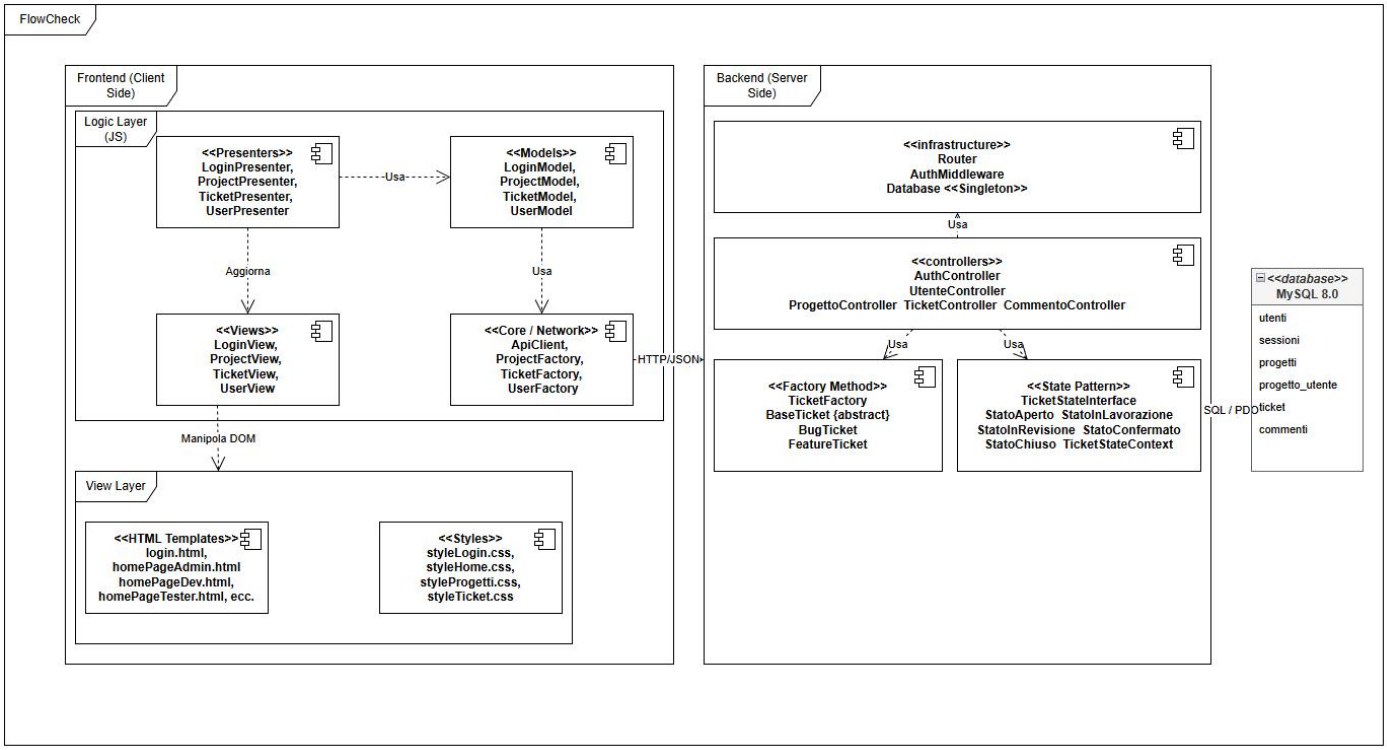
Rappresenta il flusso di lavoro per la creazione di un nuovo Ticket (feature), utilizzando le swimlanes per separare le responsabilità.



6.5 Diagramma dei Pacchetti

Organizza logicamente il sistema in sottosistemi e layer, mostrando le dipendenze macroscopiche.

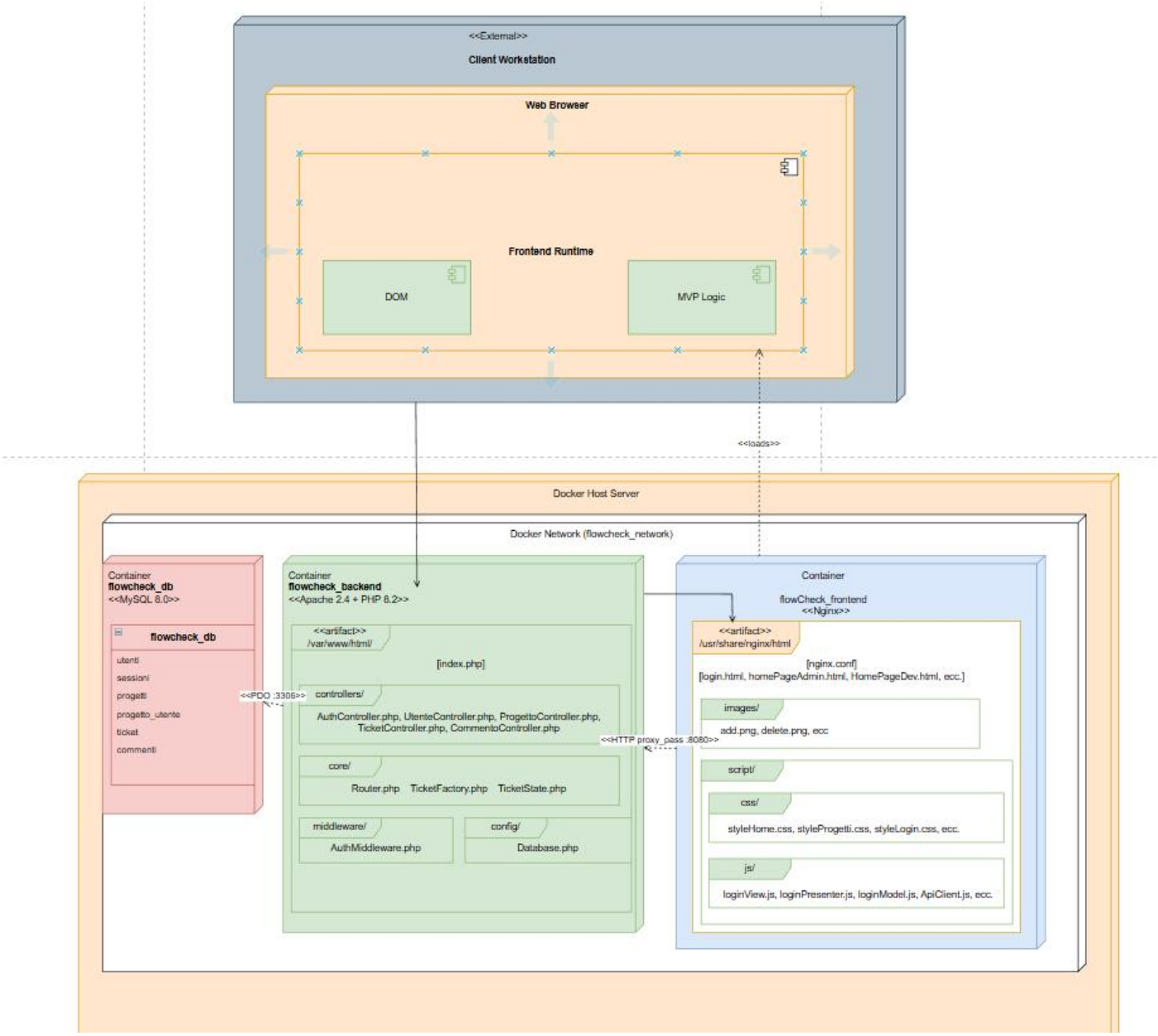
- **Frontend:** Diviso in Logic Layer (JS) e View Layer (HTML e CSS).
- **Backend:** Diviso in tre package: infrastructure (Router, AuthMiddleware, Database Singleton), controllers (AuthController, UtenteController, ProgettoController, TicketController, CommentoController) e design patterns (TicketFactory per il Factory Method e le classi StatoXxx per lo State Pattern).



6.6 Diagramma di Deployment & Componenti

Descrive l'architettura fisica di rilascio basata su container.

- **Containerizzazione:** Il sistema è basato su un Docker Host Server che esegue tre container isolati su una rete interna (flowcheck_network).
    - Frontend: Esposto sulla porta 3000.
    - Backend: Container Apache 2.4 + PHP 8.2, raggiungibile internamente sulla porta 8080; riceve le richieste proxy dal Frontend e gestisce tutte le API REST.
    - Database: accessibile solo al backend sulla porta 3306.
    - Client Workstation: accede all'applicazione tramite Browser Web



7. Testing

7.1 Metodologia

È stata adottata una strategia di Integration Testing lato backend, sviluppando in PHP dei codici
che interagiscono direttamente con il database e verificano che:

- Le query SQL e le transazioni vengono eseguite correttamente
- I vincoli di chiave esterna e le regole di eliminazione a cascata siano rispettati
- La logica di business funzioni come atteso indipendentemente dall'UI
- Ogni test apre una transazione e la annulla al termine, lasciando il DB pulito

7.2 Scenari di test

A. Autenticazione

ID Test	Funzionalità Verificata	Risultato Atteso	Esito
AU-01	Login con credenziali valide	Esito positivo della funzione di validazione	**PASS**
AU-02	Login con password errata	Accesso negato	**PASS**
AU-03	Token valido recuperato da sessioni	Token trovato associato all'email corretta	**PASS**
AU-04	Token scaduto non riconosciuto	Token ignorato, query restituisce array vuoto	**PASS**
AU-05	Logout - token eliminato da sessioni	Record del token rimosso dalla tabella	**PASS**

B. Ticket

ID Test	Funzionalità Verificata	Risultato Atteso	Esito
TK-01	Creazione Bug Ticket	Creazione corretta e validazione superata	**PASS**
TK-02	Creazione Feature Ticket senza assegnarlo	Validazione fallita	**PASS**
TK-03	Creazione Feature Ticket	Validazione superata	**PASS**

ID Test	Funzionalità Verificata	Risultato Atteso	Esito
TK-04	Creazione Ticket di un tipo inesistente	Sollevata InvalidArgumentException	**PASS**
TK-05	Transizione Ticket da "*open*" a "*progress*"	Transizione consentita	**PASS**
TK-06	Transizione Ticket da "*progress*" a "review"	Transizione consentita	**PASS**
TK-07	Chiedo se un Ticket chiuso può avanzare di stato	Metodo restituisce false	**PASS**
TK-08	Salto di stato	L'implementazione impedisce il salto	**PASS**
TK-09	Inserimento ticket	Record persistito correttamente	**PASS**
TK-10	Eliminazione del progetto contenente il ticket	Il ticket viene rimosso in automatico grazie ai vincoli ON DELETE CASCADE	**PASS**

C. Progetti

ID Test	Funzionalità Verificata	Risultato Atteso	Esito
PR-01	Creazione e lettura progetto	Progetto creato e letti attributi	**PASS**
PR-02	Aggiornamento progetto	Nuovi valori correttamente salvati nel record	**PASS**
PR-03	Tentativo di inserire un progetto creato da un utente inesistente	Il DB solleva una PDOException	**PASS**
PR-04	Visibilità progetti	Il dev vede solo progetti a lui assegnati, ignorando gli altri	**PASS**
PR-05	Eliminazione progetto	Rimuove automaticamente le associazioni al progetto e tutti i ticket	**PASS**

D. Commenti

ID Test	Funzionalità Verificata	Risultato Atteso	Esito
CM-01	Aggiunta commento	Commento recuperato correttamente dalla query	**PASS**
CM-02	Commento su ticket	Il DB solleva una	**PASS**

ID Test	Funzionalità Verificata	Risultato Atteso	Esito
	inesistente	eccezione	
CM-03	Modifica commento proprio	La query ritorna rowCount() === 1	**PASS**
CM-04	Modifica commento altrui bloccata	La query ritorna rowCount() === 0	**PASS**
CM-05	Eliminazione del ticket padre	Il commento scompare dalla tabella	**PASS**

7.3 Istruzioni per l'esecuzione dei test

Per eseguire i test, seguire i seguenti passaggi:

1. **Avvio dell'infrastruttura**: Assicurarsi che l'ambiente Docker sia in esecuzione lanciando il comando dal terminale nella directory radice del progetto: *docker compose up -d*
2. **Esecuzione e Visualizzazione**: Aprire un qualsiasi browser web e navigare all'indirizzo del backend API: *http://localhost:8080/run_all.php*
3. **Lettura dei Risultati**: Al caricamento della pagina, lo script PHP si connetterà al database MySQL, eseguirà tutti i test e restituirà una dashboard riassuntiva. La pagina mostrerà l'elenco dei test raggruppati per modulo, segnalando per ciascuno l'esito positivo (PASS) o il dettaglio dell'errore in caso di fallimento (FAIL), concludendo con un riepilogo generale.

8. Istruzioni per l'installazione

Il progetto è configurato per poter essere avviato in modo semplice e veloce grazie a Docker Compose.

8.1 Pre-requisiti:

- Docker Desktop: Necessario per poter avviare i container che gestiscono l'applicazione.

- Git / Account GitHub: Necessario per scaricare il progetto in locale.

8.2 Setup e Avvio:

1. Clonare o scaricare il repository del progetto in locale.

2. Aprire una console del terminale all'interno della cartella del progetto.

3. Digitare il comando *docker-compose up -d* per avviare il container.

Il sistema sarà accessibile all'indirizzo *http://localhost:3000*.

8.3 Credenziali Predefinite.

Al primo avvio dell'applicazione si avranno a disposizione le credenziali predefinite per l'Admin

Ruolo	**E-mail**	**Password**
Admin	Admin@flowcheck.local	password

9. Organizzazione del Team e Divisione dei Compiti

Lo sviluppo del progetto FlowCheck è stato condotto in collaborazione, adottando un approccio agile per la suddivisione delle responsabilità. Il carico di lavoro è stato distribuito in modo strategico per consentire l'avanzamento in parallelo dei moduli di Backend e Frontend, mantenendo costante una revisione del codice incrociata sul nuovo codice prodotto e un debugging congiunto di esso.

9.1 Metodologia e Processo di Sviluppo

L'applicazione è stata sviluppata seguendo un approccio iterativo, strutturato in fasi logiche per assicurare la conformità i requisiti del progetto.

1. **Fase di Analisi e Progettazione Architetturale:** In fase preliminare, il team ha analizzato i requisiti del "Progetto 12". In questa fase sono stati elaborati delle prime versioni dei diagrammi UML (Casi d'Uso e Attività), oltre che a definire le entità del database e le funzioni chiave. Inoltre, è stato configurato l'ambiente virtualizzato tramite Docker, che ha garantito a tutti i membri del team un ambiente di sviluppo condiviso e compatibile indipendentemente dalle macchine degli sviluppatori.
2. **Fase di Implementazione:** Lo sviluppo dell'applicazione è proceduto in maniera parallela su Backend e Frontend, rispettando il pattern architetturale MVP (Model-View-Presenter).
    a. Come prima cosa è stata costruito il database per gestire i dati

b. Successivamente è stata sviluppata l'interfaccia Frontend e la logica in JavaScript per sfruttare le API del Backend.
c. In parallelo al Frontend sono state sviluppate le API del Backend e i metodi necessari per gestirle.
d. La collaborazione e l'integrazione continua delle modifiche apportate dai singoli sviluppatori sono state gestite tramite l'utilizzo dei repository su GitHub.

3. **Fase di Testing:** Al termine dell'implementazione, è stata eseguita una suite di test per verificare la stabilità del sistema, portando alla versione finale del sistema.

9.2 Matrice delle responsabilità

Area Funzionale	**Responsabili**
Backend (PHP)	Andrea Marchini, Giuseppe Di Gennaro
Frontend (HTML e CSS)	Mattia Bernini
Frontend (JavaScript)	Simone Popoli, Benedetto Troni
Docker (Dockerfile & docker-compose)	Giuseppe Di Gennaro
Database	Andrea Marchini, Benedetto Troni

Area Funzionale	**Attività**	**Responsabile/i**
Project Management	Gestione Repository, Versioning	Simone Popoli
	Redazione documentazione	Congiunto
	Diagrammi UML (Frontend)	Simone Popoli, Benedetto Troni
	Diagrammi UML (Backend)	Andrea Marchini, Giuseppe Di Gennaro
Infrastruttura	Configurazione Docker	Giuseppe Di Gennaro
	Configurazione Database (database.php)	Andrea Marchini, Benedetto Troni
	Script schema.sql	Andrea Marchini, Benedetto Troni
Backend (Core & Middleware)	Router.php, TicketFactory.php, TicketState.php	Andrea Marchini
	AuthMiddleware.php	Andrea Marchini
Backend (Controllers & Config)	AuthController.php, CommentoController.php, ProgettoController.php, TicketController.php, UtenteController.php	Giuseppe Di Gennaro
Frontend (HTML & CSS)	login.html, homepageAdmin.html, homePageDev.html, ecc.	Mattia Bernini
	styleLogin.css, styleHome.css, styleTicket.css, ecc	Mattia Bernini
Frontend JS (Views & Presenter)	LoginView.js, ProjectView.js, TicketView.js, UserView.js	Simone Popoli
	LoginPresenter.js, ProjectPresenter.js, TicketPresenter.js, UserPresenter.js	Simone Popoli
	app.js	Simone Popoli
Frontend (Models & Core)	CommentModel.js, LoginModel.js, ProjectModel.js, TicketModel.js, UserModel.js	Benedetto Troni

	ApiClient.js, ProjectFactory.js, TicketFactory.js, TicketState.js, Toast.js, UserFactory.js	Benedetto Troni
Testing	AuthTest.php, CommentoTest.php, ProgettoTest.php, run_all.php, TestBase.php, TicketTest.php	Mattia Bernini
Cross-Functional	Debugging e Risoluzione Errori	Congiunto